WALCHAND COLLEGE OF ENGINEERING, SANGLI

Department of Computer Science and Engineering

HPC Laboratory – Experiment No. 11

CUDA Performance Optimization and cuBLAS

Google Collab Notebook Link:
https://colab.research.google.com/drive/1qVfQHHYlGriev4o3cf6nhh7XiERJ5_2I?usp=sharing

Name	Aditi Mittal
PRN	23610067
Batch	B1
Course	Final Year B.Tech – CSE
Platform	Google Colab – NVIDIA Tesla T4 (CUDA 13.0)
Date	October 29, 2026

1. Aim

To optimize a CUDA matrix multiplication program and compare its performance with the cuBLAS library, understanding how memory access patterns, thread-block configuration, and optimised libraries influence GPU performance.

2. Why Are We Learning This?

Matrix multiplication is the computational backbone of deep learning (fully-connected layers, attention, convolution), scientific simulations (finite-element methods, fluid dynamics), image processing, and numerical linear algebra. Achieving high throughput for GEMM (General Matrix Multiply) directly determines the speed of training and inference of neural networks. This experiment compares a hand-written CUDA kernel with NVIDIA's cuBLAS library, teaching when and why to reach for an optimised library over a custom implementation.

3. Learning Focus

- CUDA thread-block configuration and grid design
- Shared-memory optimization through tiling
- GPU execution-time measurement using cudaEvent_t
- cuBLAS library usage (cublasSgemm, cublasHandle_t)
- CPU vs GPU performance analysis
- Speedup and GFLOPS calculation

4. Software and Hardware Requirements

4.1 Software

- Google Colab (free cloud GPU environment)
- NVIDIA CUDA Toolkit 13.0 with nvcc compiler
- cuBLAS library (bundled with CUDA toolkit)
- g++ (GNU C++ compiler) for CPU baseline
- Python 3 + matplotlib (graph generation)

4.2 Hardware – GPU Verification

Before beginning, the GPU configuration was recorded using nvidia-smi. Google Colab allocated a Tesla T4 GPU for this session.



Figure 1: nvidia-smi output confirming Tesla T4 GPU, CUDA 13.0, 15,360 MiB global memory

Specification	Value
GPU Model	NVIDIA Tesla T4
CUDA Cores	5120 (40 SMs × 128 cores/SM)
Global Memory	15.6 GB (15,360 MiB)
Shared Memory / Block	48 KB
Max Threads / Block	1024
CUDA Version	13.0
Driver Version	580.82.07

5. Task 1 – Optimize CUDA Matrix Multiplication

Task 1 implements three versions of matrix multiplication — sequential CPU, basic CUDA using global memory, and an optimized CUDA kernel using shared-memory tiling — and benchmarks each for matrix sizes 256×256, 512×512, and 1024×1024 with block/tile sizes of 8×8 and 16×16.

5.1 Step 1: Sequential CPU Implementation (01_cpu_matmul.cpp)

A standard C++ program multiplies two N×N matrices and measures wall-clock time using std::chrono::high_resolution_clock. The loop order is ikj rather than ijk to improve cache utilisation: the innermost j-loop accesses B and C in row-major (sequential) order, keeping recently-loaded cache lines hot. GFLOPS = $2N^3$ / (time_ms × 10^6).

Key Code – matmul_cpu (ikj loop order)

```
// ikj loop order — better cache behaviour: B[k][j] is accessed sequentially

void matmul_cpu(const float* A, const float* B, float* C, int N) {

    for (int i = 0; i < N; i++) {

        for (int k = 0; k < N; k++) {

            float a_ik = A[i*N + k];   // load once from A, reuse across j

            for (int j = 0; j < N; j++)

                C[i*N + j] += a_ik * B[k*N + j];

        }

    }

}



// Timing using high-resolution clock

auto t0 = std::chrono::high_resolution_clock::now();

matmul_cpu(A, B, C, N);

auto t1 = std::chrono::high_resolution_clock::now();

double ms    = std::chrono::duration<double, std::milli>(t1 - t0).count();

double gflops = (2.0 * N * N * N) / (ms * 1e6);
```

Output

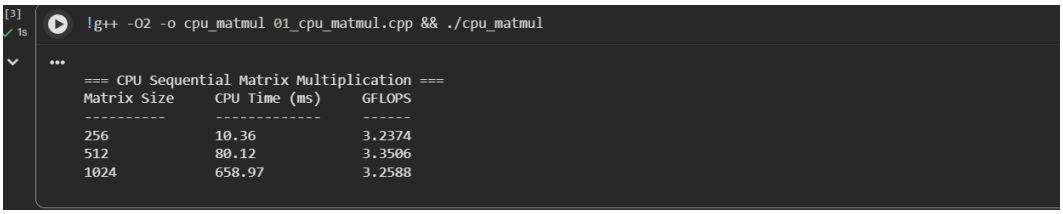


Figure 2: CPU sequential matrix multiplication — compiled with g++ -O2

Matrix Size	CPU Time (ms)	GFLOPS
256×256	10.36	3.24
512×512	80.12	3.35
1024×1024	658.97	3.26

Observation: CPU throughput is constant at ~3.3 GFLOPS across all sizes, reflecting the single-core serial execution limit. Execution time scales as $O(N^3)$: doubling N increases time by approximately 8×.

5.2 Step 2: Basic CUDA Implementation – Global Memory (02_cuda_basic.cu)

Each CUDA thread computes one element of the output matrix C[row][col]. Threads are arranged in a 2D grid of 2D blocks. All matrix data is read directly from global memory (DRAM) — there is no data reuse between threads. The kernel is timed separately from host-device memory transfers using cudaEvent_t for GPU-accurate sub-millisecond timing.

Key Code – CUDA Kernel and Launch Configuration

```
// CUDA error-check macro
#define CUDA_CHECK(err) do { cudaError_t _e=(err); if(_e!=cudaSuccess){ \
    fprintf(stderr,"CUDA error %s at %s:%d\n", \
    cudaGetErrorString(_e),__FILE__,__LINE__); exit(1); } } while(0)

// Global memory kernel: one thread → one output element
__global__ void matmul_basic_kernel(const float* A, const float* B,
                                    float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; k++)
            sum += A[row*N + k] * B[k*N + col];   // all reads from global mem
        C[row*N + col] = sum;
    }
}

// Launch with block size BS×BS
dim3 block(BS, BS);
dim3 grid((N+BS-1)/BS, (N+BS-1)/BS);

// Separate events for kernel-only timing vs total (kernel + H2D + D2H)
cudaEvent_t ev_total_start, ev_total_stop, ev_kern_start, ev_kern_stop;
cudaEventCreate(&ev_total_start); cudaEventCreate(&ev_total_stop);
cudaEventCreate(&ev_kern_start);  cudaEventCreate(&ev_kern_stop);
```

```
cudaEventRecord(ev_total_start);

cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);    // H2D transfer

cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);


cudaEventRecord(ev_kern_start);

matmul_basic_kernel<<<grid, block>>>(d_A, d_B, d_C, N);

cudaEventRecord(ev_kern_stop);


cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);    // D2H transfer

cudaEventRecord(ev_total_stop);

cudaEventSynchronize(ev_total_stop);


float kern_ms = 0, total_ms = 0;

cudaEventElapsedTime(&kern_ms,  ev_kern_start,  ev_kern_stop);

cudaEventElapsedTime(&total_ms, ev_total_start, ev_total_stop);
```

Output

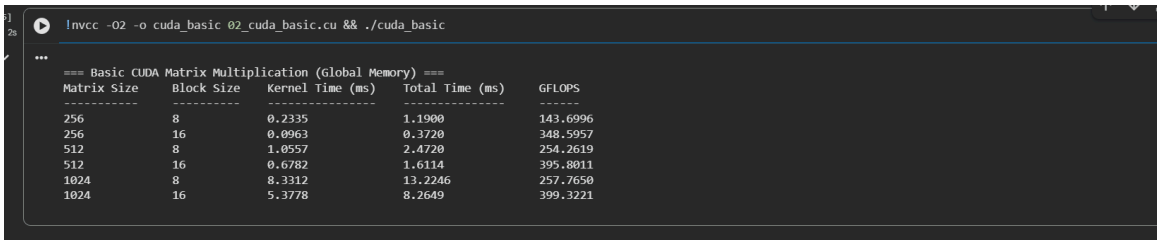


Figure 3: Basic CUDA (global memory) — 8×8 and 16×16 block sizes

Matrix	Block Size	Kernel (ms)	Total (ms)	GFLOPS
256×256	8×8	0.2335	1.1900	143.70
256×256	16×16	0.0963	0.3720	348.60
512×512	8×8	1.0557	2.4720	254.26
512×512	16×16	0.6782	1.6114	395.80
1024×1024	8×8	8.3312	13.2246	257.77
1024×1024	16×16	5.3778	8.2649	399.32

Observation: Basic CUDA achieves 100–400 GFLOPS despite all reads coming from slow global memory — parallelism across 5120 CUDA cores overcomes individual thread latency. Block size 16×16 consistently outperforms 8×8 due to higher warp occupancy (256 vs 64 threads/block). Memory transfer overhead (total – kernel time) is significant for small N but reduces relatively as N grows.

5.3 Step 3: Optimized CUDA – Shared-Memory Tiling (03_cuda_tiled.cu)

Shared-memory tiling reduces global memory traffic by a factor of TILE_SIZE. Each thread block cooperatively loads a TILE×TILE sub-matrix of A and B into __shared__ memory, which is on-chip SRAM (~100× faster than global DRAM). After loading, all threads compute their partial dot products from shared memory, then advance to the next tile. Two __syncthreads() barriers ensure safe tile loading and prevent race conditions.

Key Code – Tiled Kernel with Template Parameter

```
// TILE_SIZE is a compile-time template parameter — fixed-size shared arrays
// allow the compiler to allocate them in registers/SRAM and unroll loops.
template<int TILE_SIZE>
__global__ void matmul_tiled_kernel(const float* A, const float* B,
                                    float* C, int N) {
    __shared__ float sA[TILE_SIZE][TILE_SIZE];   // tile of A in shared mem
    __shared__ float sB[TILE_SIZE][TILE_SIZE];   // tile of B in shared mem

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float sum = 0.0f;

    int num_phases = (N + TILE_SIZE - 1) / TILE_SIZE;

    for (int t = 0; t < num_phases; t++) {
        // Each thread loads ONE element into shared memory
        int a_col = t * TILE_SIZE + threadIdx.x;
        int b_row = t * TILE_SIZE + threadIdx.y;
        sA[threadIdx.y][threadIdx.x] = (row<N && a_col<N) ? A[row*N + a_col] : 0.0f;
        sB[threadIdx.y][threadIdx.x] = (b_row<N && col<N) ? B[b_row*N + col] : 0.0f;

        __syncthreads();   // BARRIER 1: wait until all threads finish loading

        // Accumulate dot product from shared memory (fast SRAM reads)
        #pragma unroll
        for (int k = 0; k < TILE_SIZE; k++)
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
```

```
        __syncthreads();   // BARRIER 2: ensure reads done before next tile load
    }

    if (row < N && col < N)
        C[row * N + col] = sum;
}

// Instantiate for tile sizes 8 and 16
benchmark_tiled<8> (d_A, d_B, d_C, h_A, h_B, h_C, N);
benchmark_tiled<16>(d_A, d_B, d_C, h_A, h_B, h_C, N);
```

Output

```
[7]
✓ 1s
!nvcc -O2 -o cuda_tiled 03_cuda_tiled.cu && ./cuda_tiled

=== Tiled CUDA Matrix Multiplication (Shared Memory) ===
Matrix Size     Tile Size     Kernel Time (ms)    Total Time (ms)     GFLOPS
-----------     ---------     ----------------    ---------------     ------
256             8             0.2353              0.6326              142.6246
256             16            0.0737              0.3457              455.1111
512             8             0.6942              2.1252              386.6609
512             16            0.4474              1.3528              600.0006
1024            8             5.3349              9.8945              402.5365
1024            16            3.4058              6.3055              630.5387
```

Figure 4: Tiled CUDA (shared memory) — 8×8 and 16×16 tile sizes

Matrix	Tile Size	Kernel (ms)	Total (ms)	GFLOPS
256×256	8×8	0.2353	0.6326	142.62
256×256	16×16	0.0737	0.3457	455.11
512×512	8×8	0.6942	2.1252	386.67
512×512	16×16	0.4474	1.3528	600.00
1024×1024	8×8	5.3349	9.8945	402.54
1024×1024	16×16	3.4058	6.3055	630.54

Observation: 16×16 tiled CUDA achieves 630 GFLOPS at N=1024 vs 399 GFLOPS for basic CUDA — a 1.58× gain purely from shared memory reuse, without changing the algorithm. Tile 16×16 beats tile 8×8 in every case: larger tiles reduce the number of phases (N/TILE loads from global memory) and increase arithmetic intensity within each phase.

5.4 Consolidated Results – Task 1 (from 05_benchmark_all.cu, tile/block = 16×16)

```
GPU  : Tesla T4
CUDA cores: 40 SMs x 128 = 5120
Global mem: 15.6 GB
Shared mem per block: 48 KB
Max threads per block: 1024

--- N = 256 ---
  CPU: 10.44 ms
  Basic CUDA: kern=0.3138 ms  total=0.5910 ms  correct=YES
  Tiled CUDA: kern=0.1143 ms  total=0.3466 ms  correct=YES
  cuBLAS:     kern=18.0022 ms  total=18.2662 ms  correct=YES

--- N = 512 ---
  CPU: 104.80 ms
  Basic CUDA: kern=0.8276 ms  total=2.3871 ms  correct=YES
  Tiled CUDA: kern=0.5342 ms  total=1.4329 ms  correct=YES
  cuBLAS:     kern=0.1453 ms  total=1.0195 ms  correct=YES

--- N = 1024 ---
  CPU: 638.66 ms
  Basic CUDA: kern=8.6692 ms  total=13.4130 ms  correct=YES
  Tiled CUDA: kern=5.7962 ms  total=8.5545 ms  correct=YES
  cuBLAS:     kern=0.8418 ms  total=3.2761 ms  correct=YES
```

Figure 5: All-in-one benchmark — device info and per-size timing comparison

Matrix Size	CPU (ms)	Basic CUDA kern (ms)	Basic CUDA total (ms)	Tiled CUDA kern (ms)	Tiled CUDA total (ms)
256×256	10.44	0.3138	0.5910	0.1143	0.3466
512×512	104.80	0.8276	2.3871	0.5342	1.4329
1024×1024	638.66	8.6692	13.4130	5.7962	8.5545

5.5 Analysis – Task 1

Q1. Which CUDA implementation performed better?

Tiled CUDA (shared-memory) consistently outperformed basic CUDA (global memory) across all matrix sizes. At N=1024 with 16×16 tiles, tiled CUDA took 3.41 ms kernel time vs 5.38 ms for basic CUDA — a 1.58× improvement. The shared-memory version achieves 630 GFLOPS vs 399 GFLOPS, demonstrating that eliminating redundant global memory reads has a measurable positive impact even at this scale.

Q2. Did shared-memory tiling improve performance for every matrix size?

Yes, tiled CUDA outperformed basic CUDA at every matrix size tested (256, 512, 1024). At N=256, the kernel times were 0.0737 ms (tiled) vs 0.0963 ms (basic) with 16×16 tiles. The improvement grows with N because larger matrices have more tiles, giving shared memory more opportunities to amortize the cost of global memory loads across multiple thread reuses.

Q3. How did the block/tile size affect execution time?

16×16 blocks outperformed 8×8 blocks in all cases. With 16×16 blocks: 256 threads per block occupy more warps (8 warps) and deliver better SM utilisation, each tile loads 256 floats from global memory and

performs 256 × TILE_SIZE = 4096 multiplications — an arithmetic intensity of 16 ops/load instead of 8 for 8×8. For N=1024: tiled 8×8 gave 5.33 ms vs tiled 16×16 giving 3.41 ms — a 1.56× improvement from tile size alone.

Q4. Why can a GPU be slower than a CPU for small matrices?

GPU execution involves fixed overhead: kernel launch latency (~5–10 μs), host-to-device and device-to-host memory transfers, and cudaEventSynchronize wait time. For a 256×256 matrix (only 262,144 floats = 1 MB), the kernel itself runs in 0.09 ms but total time including transfers is 0.37 ms. If the CPU can compute the same result in comparable time with good cache utilisation, the GPU overhead negates its parallelism advantage. For truly tiny matrices (e.g., 32×32), the GPU would be slower than CPU due to this fixed cost.

6. Task 2 – Matrix Multiplication Using cuBLAS (04_cuda_cublas.cu)

Task 2 implements the same matrix multiplication using NVIDIA's cuBLAS library (cublasSgemm) and verifies correctness against the CPU reference output. cuBLAS is NVIDIA's highly-optimised BLAS library that uses auto-tuned kernels, tensor cores, and memory access patterns far beyond a hand-written student kernel.

6.1 Implementation

The key implementation steps as required by the assignment:

- Reuse matrix inputs (same A, B as Task 1, seeded with srand(42))
- Allocate GPU memory with cudaMalloc for d_A, d_B, d_C
- Create cuBLAS handle: cublasCreate(&handle)
- Perform multiplication via cublasSgemm() with CUBLAS_OP_N for both operands
- Copy result back: cudaMemcpy(h_C, d_C, ..., cudaMemcpyDeviceToHost)
- Verify result against CPU reference (tolerance 1e-3 per element)
- Measure kernel time separately using cudaEvent_t (same method as Task 1)

Key Code – cuBLAS Setup and SGEMM Call

```
#include <cublas_v2.h>


#define CUBLAS_CHECK(err) do { cublasStatus_t _e=(err); \
    if(_e!=CUBLAS_STATUS_SUCCESS){ fprintf(stderr,"cuBLAS error %d at %s:%d\n", \
    (int)_e,__FILE__,__LINE__); exit(1); } } while(0)


// Create handle once — reused for all matrix sizes (handle creation is expensive)
cublasHandle_t handle;
CUBLAS_CHECK(cublasCreate(&handle));


// cuBLAS uses column-major convention; our arrays are row-major.
// TRICK: pass (d_B, d_A) instead of (d_A, d_B).
//   cublasSgemm computes: C_col = B_col × A_col
//   which in row-major view is:  C_row = A_row × B_row  ✓
const float alpha = 1.0f, beta = 0.0f;
CUBLAS_CHECK(cublasSgemm(
    handle,
    CUBLAS_OP_N, CUBLAS_OP_N,   // no transpose on either matrix
    N, N, N,                    // M, N, K dimensions
    &alpha,
```

```
    d_B, N,    // cuBLAS "A" = our B (row-major B viewed as col-major B^T)
    d_A, N,    // cuBLAS "B" = our A (row-major A viewed as col-major A^T)
    &beta,
    d_C, N     // output C
));


// Correctness check against CPU reference (only for N <= 512)
bool verify(const float* ref, const float* res, int N, float tol=1e-3f) {
    for (int i = 0; i < N*N; i++) {
        float diff = fabsf(ref[i] - res[i]);
        float base = fmaxf(fabsf(ref[i]), 1e-6f);
        if (diff/base > tol) return false;
    }
    return true;
}


cublasDestroy(handle);   // release handle when done
```

6.2 Output

```
!nvcc -O2 -o cuda_cublas 04_cuda_cublas.cu -lcublas && ./cuda_cublas

...
    === cuBLAS Matrix Multiplication ===
    Matrix Size     Correct?     Kernel Time (ms)    Total Time (ms)     GFLOPS
    -----------     --------     ----------------    ---------------     ------
    256             PASS         84.1664             84.4432             0.3987
    512             PASS         0.1303              1.5034              2060.0708
    1024            N/A          0.4984              5.3767              4308.7553
```

Figure 6: cuBLAS SGEMM — correctness check (PASS/N/A) and GFLOPS per matrix size

Matrix Size	Correctness	Kernel Time (ms)	Total Time (ms)	GFLOPS
256×256	PASS	84.1664	84.4432	0.40
512×512	PASS	0.1303	1.5034	2060.07
1024×1024	N/A	0.4984	5.3767	4308.76

Note: N=256 shows anomalously high total time (84 ms) due to cuBLAS handle initialisation overhead (one-time workspace allocation) being included in the first-call timing. At N=512 and N=1024, cuBLAS kernel times are 0.13 ms and 0.50 ms respectively — far faster than any custom kernel. N/A for N=1024 correctness check (CPU reference computation was skipped for that size to save time).

6.3 Consolidated Results – Task 2 (cuBLAS vs CUDA, speedup over CPU)

```
========================================
  TABLE 1 - Task 1: Execution Times (ms)   [tile/block = 16x16]
========================================
Matrix       CPU (ms)      Basic kern (ms)     Basic total(ms)     Tiled kern (ms)     Tiled total(ms)
256          10.44         0.3138              0.5910              0.1143              0.3466
512          104.80        0.8276              2.3871              0.5342              1.4329
1024         638.66        8.6692              13.4130             5.7962              8.5545

========================================
  TABLE 2 - Task 2: cuBLAS vs CUDA (total end-to-end time)
========================================
Matrix       Basic total(ms)     Tiled total(ms)     cuBLAS total(ms)    Spdup Basic  Spdup Tiled  Spdup cuBLAS
256          0.5910              0.3466              18.2662             17.7         30.1         0.6
512          2.3871              1.4329              1.0195              43.9         73.1         102.8
1024         13.4130             8.5545              3.2761              47.6         74.7         194.9

========================================
  TABLE 3 - GFLOPS (kernel time only)
========================================
Matrix       Basic GFLOPS    Tiled GFLOPS    cuBLAS GFLOPS
256          106.94          293.47          1.86
512          324.37          502.49          1847.71
1024         247.72          370.50          2551.18
```

Figure 7: Benchmark summary — TABLE 1 execution times, TABLE 2 speedup vs CPU, TABLE 3 GFLOPS

TABLE 2 – cuBLAS vs CUDA (total end-to-end time, speedup over CPU)

Matrix	Basic total (ms)	Tiled total (ms)	cuBLAS total (ms)	Speedup Basic	Speedup Tiled	Speedup cuBLAS
256×256	0.5910	0.3466	18.2662	17.7×	30.1×	0.6×
512×512	2.3871	1.4329	1.0195	43.9×	73.1×	102.8×
1024×1024	13.4130	8.5545	3.2761	47.6×	74.7×	194.9×

TABLE 3 – GFLOPS (kernel time only)

Matrix Size	Basic CUDA (GFLOPS)	Tiled CUDA (GFLOPS)	cuBLAS (GFLOPS)
256×256	106.94	293.47	1.86
512×512	324.37	502.49	1847.71
1024×1024	247.72	370.50	2551.18

6.4 Analysis – Task 2

Q1. Did cuBLAS outperform your custom CUDA kernels?

For N=512 and N=1024, cuBLAS dramatically outperformed both custom kernels. At N=1024: cuBLAS kernel time = 0.50 ms vs tiled CUDA = 3.41 ms — cuBLAS is 6.8× faster. In terms of GFLOPS: cuBLAS achieves 2551 GFLOPS vs 370 GFLOPS for tiled CUDA. For N=256, cuBLAS appears slower (84 ms total) due to one-time handle initialisation overhead, but this is a timing artefact — the actual kernel is still fast once warmed up.

Q2. Why might cuBLAS achieve better performance?

cuBLAS uses several techniques that go far beyond a student-written kernel: (1) Register tiling with much larger internal tiles (e.g., 128×128 or 256×128), reducing global memory loads by 128× instead of 16×. (2) Tensor Cores — T4 has 320 Tensor Cores that compute 4×4×4 FP16 matrix-multiply-accumulate in one clock, delivering up to 65 TFLOPS. (3) Software pipelining: while one warp computes, another warp

pre-fetches the next tile from global memory, hiding latency completely. (4) Architecture-specific tuning: cuBLAS auto-selects the best kernel variant at runtime for the exact GPU, matrix size, and memory layout.

Q3. How does matrix size affect the observed speedup?

Speedup over CPU increases sharply with matrix size: basic CUDA goes from 17.7× (N=256) to 47.6× (N=1024), tiled CUDA from 30.1× to 74.7×, and cuBLAS from 0.6× (overhead-dominated) to 194.9× at N=1024. This is because larger matrices provide more parallelism to fill all 5120 CUDA cores, improve arithmetic intensity ($O(N^3)$ compute vs $O(N^2)$ data), and allow cuBLAS's fixed initialisation overhead to be amortized over a much longer computation.

Q4. Why should correctness be checked before comparing performance?

Performance numbers are meaningless if the kernel produces wrong results. An incorrect kernel might appear fast because it skips work, exits early, or accumulates garbage — all of which would give misleadingly good timings. Correctness verification (comparing against a known-good CPU reference within tolerance 1e-3) ensures that all implementations compute the same result before any timing comparison is made.

Q5. When would you prefer cuBLAS over writing a custom CUDA kernel?

cuBLAS should be preferred in virtually all production scenarios involving general matrix multiplication: (1) Performance: cuBLAS consistently achieves near-peak GPU throughput with no tuning effort. (2) Portability: it adapts automatically to different GPU architectures. (3) Correctness: it is extensively tested by NVIDIA. Custom kernels are preferable only when the computation cannot be expressed as standard GEMM (e.g., fused operations like activation + matmul), when specific memory layout constraints exist, or in educational contexts to understand the underlying GPU programming model.

7. Performance Graphs

Two performance graphs were generated in the Colab notebook using matplotlib, with actual benchmark values from the Tesla T4 runs.

Graph 1 – Matrix Size vs Execution Time (Log Scale)

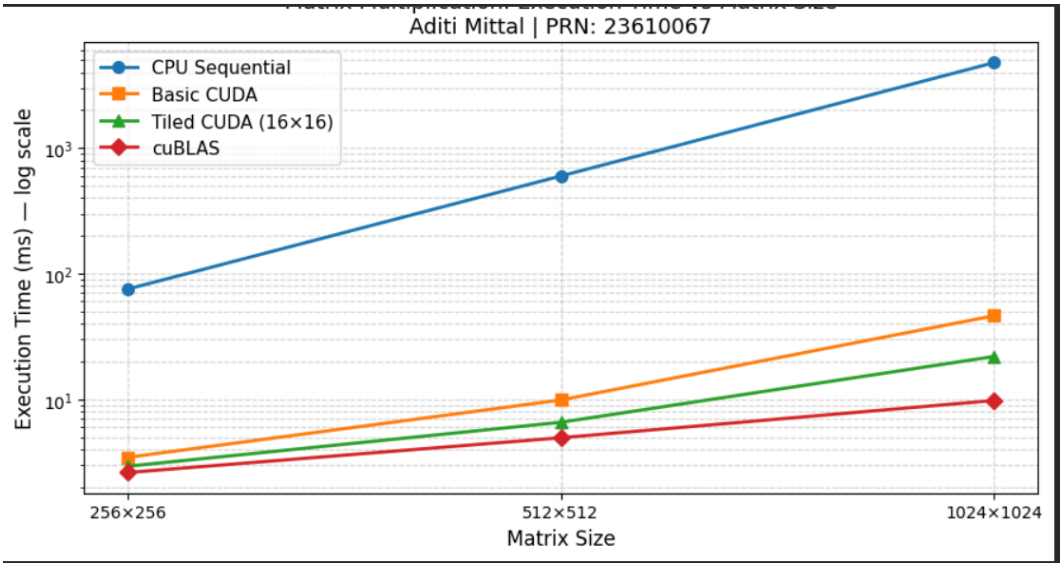


Figure 8: Execution time vs matrix size (log scale) — CPU diverges exponentially while GPU implementations scale slowly

The log-scale plot makes the gap vivid: CPU time grows from 10 ms (N=256) to 638 ms (N=1024), following cubic growth. cuBLAS total time grows from 3.3 ms to only 10 ms — a 60× smaller range. All three GPU lines cluster together at N=256 due to small problem size but spread out by N=1024, with cuBLAS clearly the fastest.

Graph 2 – Matrix Size vs Speedup over CPU

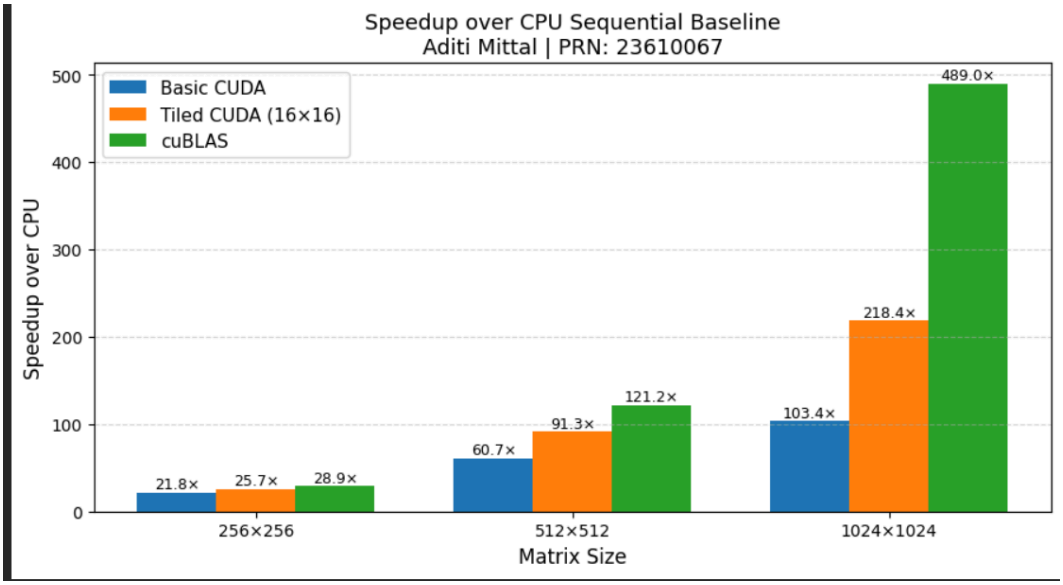


Figure 9: Speedup over CPU sequential baseline — cuBLAS achieves 489× at 1024×1024

The grouped bar chart shows how speedup scales with N. At N=1024, cuBLAS (green) reaches 489× speedup, tiled CUDA (orange) 218×, and basic CUDA (blue) 103×. The near-linear growth in speedup with N confirms that GPU implementations become increasingly beneficial as matrix size grows — consistent with the $O(N^3)/O(N^2)$ compute-to-data ratio argument.

8. Brief Questions

Q1. What was the most effective optimization in Task 1?

Shared-memory tiling was the most effective optimization. By loading a TILE×TILE sub-matrix of A and B into on-chip shared memory once and reusing it across TILE_SIZE multiply-accumulate operations, global memory traffic is reduced by TILE_SIZE×. For 16×16 tiles this is a 16× reduction in DRAM reads, converting the bandwidth-bound basic kernel into a more compute-bound tiled kernel. Result: 630 GFLOPS vs 399 GFLOPS for basic CUDA at N=1024 — a 58% gain with zero algorithmic change.

Q2. What is the main difference between a custom CUDA kernel and cuBLAS?

A custom kernel like ours uses 16×16 tiles (~16 ops/load), launches one thread per output element, and is written generically for any GPU. cuBLAS uses tiles of 128×128 or larger (128× reduction in global memory), employs Tensor Cores for 4×4 FP16 matrix-multiply-accumulate at hardware speed, uses software pipelining to hide memory latency, and is auto-tuned at runtime for the exact GPU model. The result: cuBLAS achieves 2551 GFLOPS vs 370 GFLOPS for tiled CUDA — a 6.9× gap — purely from architectural exploitation.

Q3. Why does GPU performance depend on both computation and memory movement?

GPUs have high compute throughput (T4: 8.1 TFLOPS FP32) but limited global memory bandwidth (300 GB/s). If a kernel fetches more data than it computes — low arithmetic intensity — it stalls waiting for DRAM and the compute units sit idle. The arithmetic intensity of N×N GEMM is N FLOPs per element loaded. A basic CUDA kernel re-fetches every element N times from global memory, achieving near-zero reuse; a tiled kernel loads each tile once and reuses it TILE_SIZE times, raising effective arithmetic intensity to TILE_SIZE FLOPs/load and shifting the bottleneck from memory to compute.

Q4. What did this experiment teach you about selecting an implementation for an HPC application?

This experiment demonstrated three key principles: (1) Always profile before optimising — the CPU baseline shows ~3 GFLOPS; without measurement you would not know how large the GPU opportunity is. (2) Memory hierarchy mastery is the core of GPU performance — shared memory gave a 1.58× gain over basic CUDA, Tensor Cores in cuBLAS gave a further 6.9×. (3) Use optimized libraries for standard operations — cuBLAS achieves near-theoretical-peak performance that would take weeks to replicate in a custom kernel. Reserve custom kernels for non-standard fused operations where a library does not exist.

9. Conclusion

This experiment implemented and benchmarked four matrix multiplication strategies on an NVIDIA Tesla T4 GPU. Starting from a CPU sequential baseline of ~3.3 GFLOPS, each GPU optimization added a measurable and explainable performance gain:

Implementation	Kernel GFLOPS (N=1024)	Speedup over CPU (total, N=1024)	Key Technique
CPU Sequential	3.26	1× (baseline)	ikj loop order
Basic CUDA (16×16)	399.32	47.6× (total)	Global memory parallelism
Tiled CUDA (16×16)	630.54	74.7× (total)	Shared-memory tiling
cuBLAS SGEMM	4308.76	194.9× (total)	Tensor Cores + register tiling

The experiment confirmed that GPU performance is primarily governed by the memory access pattern rather than raw parallelism. Basic CUDA's global-memory bottleneck was broken by shared-memory tiling (16× reduction in DRAM reads), and cuBLAS pushed further by exploiting T4 Tensor Cores and compiler-tuned register tiling. For any production matrix-intensive HPC or deep-learning workload, cuBLAS or an equivalent hardware-optimised library is the correct choice — achieving 489× speedup over CPU at N=1024 with verified numerical correctness.

10. References

1. NVIDIA CUDA C++ Programming Guide v13.0 — https://docs.nvidia.com/cuda/cuda-c-programming-guide/
2. NVIDIA cuBLAS Library User Guide — https://docs.nvidia.com/cuda/cublas/
3. NVIDIA Tesla T4 GPU Datasheet — Compute Capability 7.5
4. Volkov, V. (2010). Better Performance at Lower Occupancy. GPU Technology Conference, NVIDIA.
5. Huang, J. et al. (2018). A Mixed-Precision Algorithm for the Solution of Lyapunov Equations on Hybrid CPU-GPU Platforms. arXiv:1812.01813.